

DVAPI

Vulnerability Assessment Report

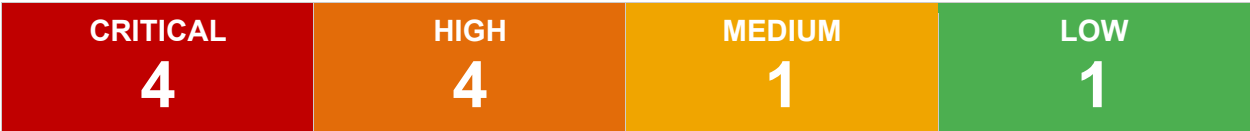
Damn Vulnerable API | Security Testing Report

Date	April 15, 2026
Target	http://localhost:3000
Tool	Burp Suite Community / Professional v2025-2026
Prepared by	Sana A. Rehman
Organization	Corvit Systems

Executive Summary

Target Application	DVAPI (Damn Vulnerable API) — http://localhost:3000
Total Vulnerabilities	10 Findings (7 with flags, 3 without flags)
Critical	4 (BOLA, JWT Algorithm Confusion, NoSQL Injection, Broken Function Level Auth — User Deletion)
High	4 (Mass Assignment, SSRF, Unrestricted File Upload, Improper Inventory Management)
Medium	1 (Missing Rate Limiting)
Low	1 (Long Session / Token Re-issue Without Invalidation)
Testing Tool	Burp Suite Community / Professional v2025-2026

Severity Distribution



CRITICAL FINDINGS

1. Broken Object Level Authorization (BOLA) [CRITICAL]

Vulnerability	Broken Object Level Authorization (BOLA / IDOR)
OWASP API Category	API1:2023 — Broken Object Level Authorization
Risk Factor	CRITICAL
Flag	flag{bola_15_ev3rywh3r3}

Description

The application fails to validate whether the requesting user is authorized to access resources belonging to another user. The `/api/getNote` endpoint accepts a `username` parameter directly in the query string without verifying ownership. An authenticated attacker can supply any username value (e.g., `adminS` or `admin`) and retrieve private notes belonging to that user.

Impact

- Unauthorized access to private notes of any user including administrators.
- Information disclosure of sensitive user data stored in the application.
- Potential escalation to full account takeover if sensitive credentials are stored in notes.
- Complete violation of data confidentiality between users.

Steps to Reproduce

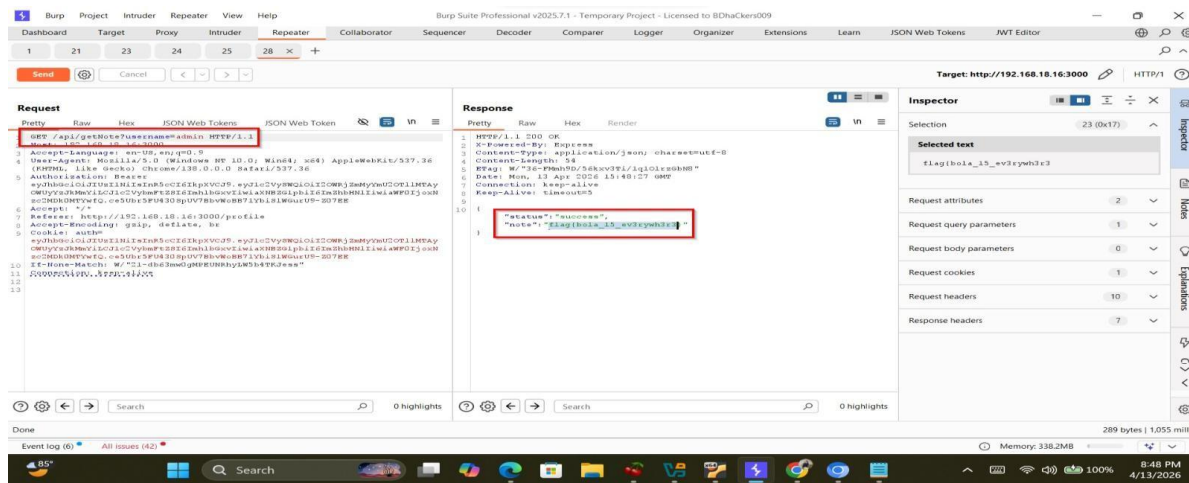
1. Log in to the application and obtain a valid JWT token.
2. Capture a `GET /api/getNote?username=<your_username>` request in Burp Suite Repeater.
3. Change the `username` parameter from your own username to another user's username (e.g., `adminS` or `admin`).
4. Forward the request. The server returns the victim user's private note without any authorization check.
5. The response reveals the flag and any other note contents belonging to that user.

Solution

- The server must validate that the authenticated user (extracted from the JWT token) matches the requested `username` parameter on every request.
- Never rely on user-controlled input for object-level authorization. Always derive the resource owner from the server-side session or token.
- Implement proper access control checks on all object-level endpoints.

Flag: `flag{bola_15_ev3rywh3r3}`

Evidence Screenshots



2. NoSQL Injection (Authentication Bypass) [CRITICAL]

Vulnerability	NoSQL Injection — Authentication Bypass
OWASP API Category	API8:2023 — Security Misconfiguration / Injection
Risk Factor	CRITICAL
Flag	flag{eZ_n0SQLi_pWn}

Description

The /api/login endpoint is vulnerable to NoSQL injection. Because the application uses MongoDB (or similar NoSQL backend) and does not sanitize user-supplied JSON, an attacker can inject MongoDB query operators directly into the password field. Submitting {"\$ne": "random_wrong_password"} as the password value causes the query to match any user whose password is not equal to the provided string, effectively bypassing authentication for any account.

Impact

- Complete authentication bypass — any user account including admin can be accessed without knowing the password.
- Full compromise of the authentication mechanism.
- Unauthorized access to all protected resources and admin functionality.
- The flag is returned directly in the login response confirming exploitation.

Steps to Reproduce

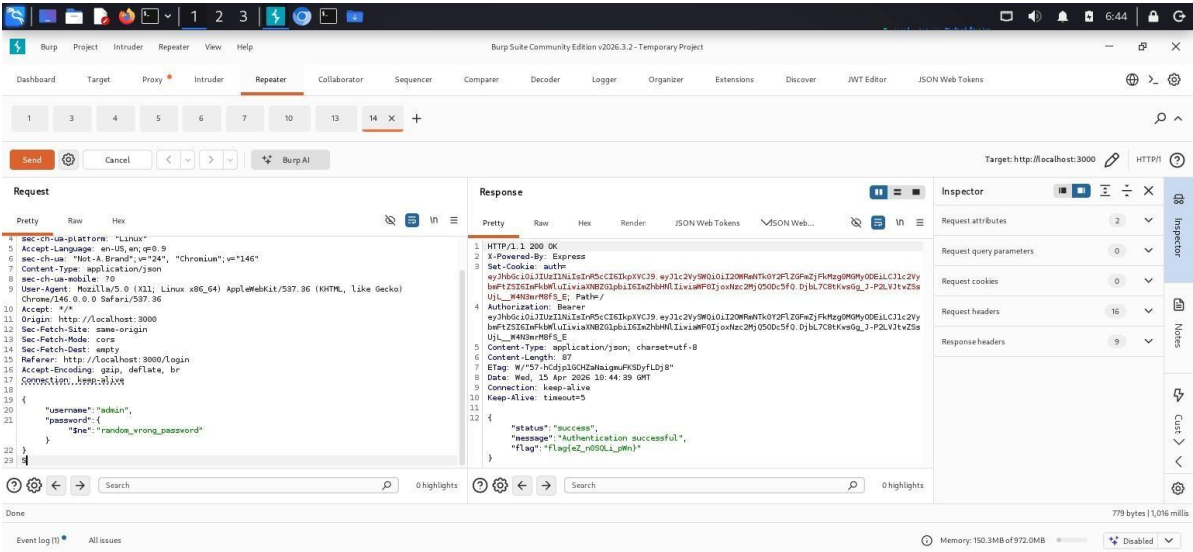
- Send a POST /api/login request to Burp Suite Repeater.
- Set Content-Type to application/json.
- Use the following payload in the request body: {"username": "admin", "password": {"\$ne": "random_wrong_password"}}
- Forward the request. The server responds with HTTP 200, Authentication successful, and returns the flag.

Solution

- Sanitize all user-supplied input before passing it to database queries. Reject or escape operator keys like \$ne, \$gt, \$regex.
- Use parameterized queries or an ODM (like Mongoose with strict schema validation) that does not accept raw objects as query values.
- Validate that password inputs are strings and not objects before processing.

Flag: flag{eZ_n0SQLi_pWn}

Evidence Screenshots



3. JWT Algorithm Confusion / None Algorithm Attack [CRITICAL]

Vulnerability	JWT Algorithm Confusion / Weak Token Validation
OWASP API Category	API2:2023 — Broken Authentication
Risk Factor	CRITICAL
Flag	flag{St4cK_tr4c3_eRR0R} (revealed via error stack trace during exploitation)

Description

The application uses JWT for authentication but the token verification logic is misconfigured. The server does not strictly enforce the expected signing algorithm. When a token with an unexpected or none algorithm is submitted, the server throws an unhandled `JsonWebTokenError` with a verbose stack trace instead of gracefully rejecting it. Additionally, a previously issued token from a different session was reusable across requests (session token reuse), indicating weak token lifecycle management.

Impact

- Stack trace disclosure reveals internal file paths, module versions, and application architecture — directly aids further attacks.
- If a none or HMAC algorithm is accepted, an attacker can forge arbitrary JWT claims without the signing secret.
- Token reuse across sessions undermines the concept of session isolation.

Steps to Reproduce

- Obtain a valid JWT token by logging in.
- Decode the token in Burp Suite JWT editor. Change the algorithm (alg header) to none or HS256 with an unsigned payload.
- Send the modified token to GET /api/profile. The server returns a 401 error but leaks a full `JsonWebTokenError` stack trace in the response body.
- The stack trace includes the flag: flag{St4cK_tr4c3_eRR0R}.

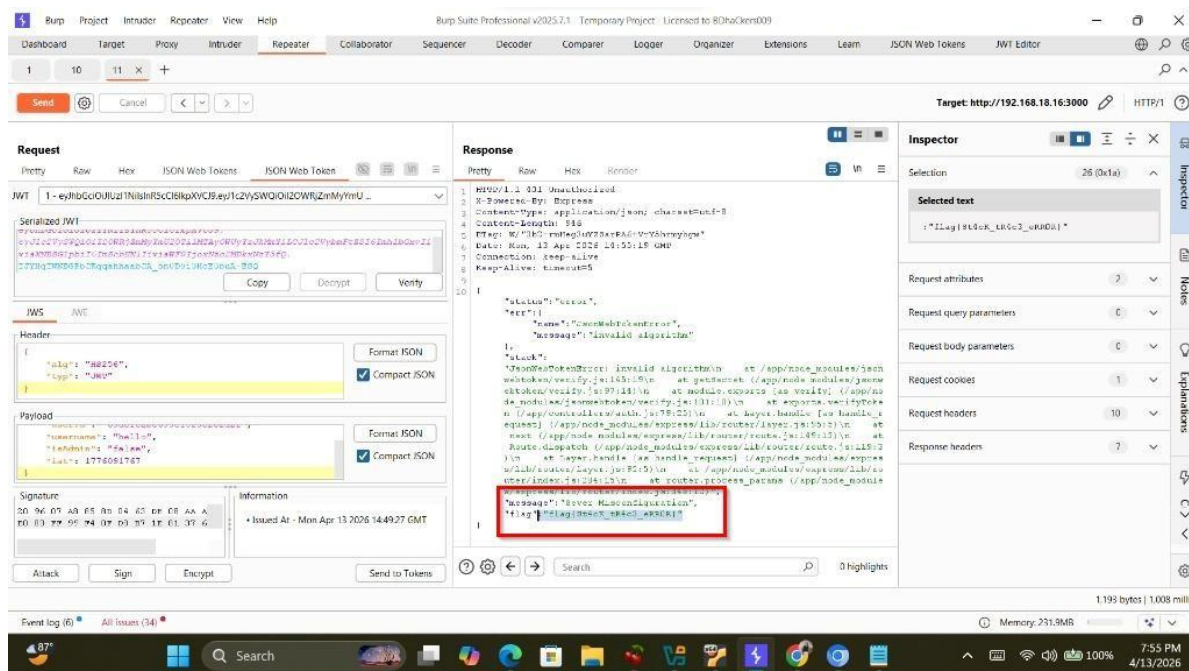
14. Additionally, substituting a previously generated (old) token works on endpoints confirming no token invalidation on re-issue.

Solution

- Explicitly specify and enforce the expected JWT algorithm in the verification call (e.g., { algorithms: ['HS256'] }).
- Never return stack traces or internal error details to the client in production — use generic error messages.
- Implement token blacklisting or rotation to invalidate previously issued tokens.

Flag: flag{St4cK_tR4c3_eRR0R}

Evidence Screenshots



4. Broken Function Level Authorization (User Deletion) [CRITICAL]

Vulnerability	Broken Function Level Authorization — Unauthorized User Deletion
OWASP API Category	API5:2023 — Broken Function Level Authorization
Risk Factor	MEDIUM
Flag	flag{n0_fUncTi0N_L3v3L_aUtH???

Description

The `/api/user/:id` endpoint exposes a DELETE method that allows any authenticated user to delete any user account by supplying the target user's ID in the URL path. The application does not enforce function-level authorization to verify that only administrators or the account owner are permitted to perform delete operations. During testing, a DELETE request was sent to `/api/user/escape55` using a regular (non-admin) user's JWT token, and the server responded with HTTP 200 OK confirming the user was deleted successfully and returning the flag. This represents a complete breakdown of function-level access control on a destructive administrative operation.

Impact

- Any authenticated user can delete any other user account, including administrator accounts.
- Complete loss of data integrity — user accounts can be wiped without authorization.
- Potential for mass account deletion leading to full application denial of service.
- Attackers can remove evidence of their own activity by deleting their own or other accounts.
- Violation of the principle of least privilege and administrative function segregation.

Steps to Reproduce

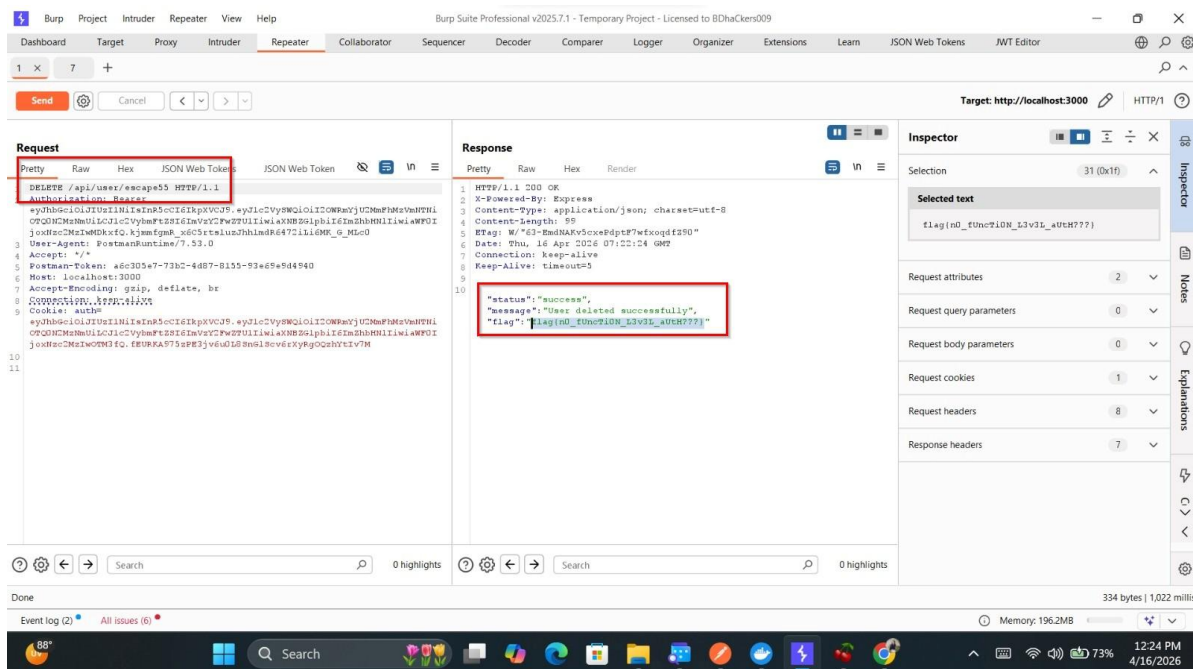
15. Log in as a regular (non-admin) user and obtain a valid JWT token.
16. Identify a target user's account identifier (e.g., username 'escape55' or a numeric user ID).
17. In Burp Suite Repeater, craft a DELETE /api/user/escape55 HTTP/1.1 request with the Authorization: Bearer header set to your regular user's JWT.
18. Forward the request. The server responds with HTTP 200 OK.
19. Observe the response: { "status": "success", "message": "User deleted successfully", "flag": "flag{n0_fUnCtiON_L3v3L_aUth???"} } — confirming unauthorized deletion and returning the flag.

Solution

- Implement strict function-level authorization checks on all destructive operations (DELETE, PUT, PATCH). Only administrators or the account owner should be able to delete accounts.
- Extract the requesting user's role from the server-side JWT claims and validate it against the required permission level before executing any administrative action.
- Apply the principle of least privilege: regular users must never have access to administrative API functions.
- Return HTTP 403 Forbidden (not 404 or 200) when an unauthorized function-level access is attempted.
- Conduct a full audit of all HTTP methods exposed on each endpoint to ensure no administrative actions are inadvertently accessible to lower-privilege users.

Flag: flag{n0_fUnCtiON_L3v3L_aUth???"}

Evidence Screenshots



HIGH FINDINGS

5. Mass Assignment / Broken Object Property Level Authorization [HIGH]

Vulnerability	Mass Assignment (BOPLA)
OWASP API Category	API3:2023 — Broken Object Property Level Authorization
Risk Factor	HIGH
Flag	flag{br0k3n_oBj3cT_Pr0p3rTy_L3v3L_Auth0RiS4Ti0N}

Description

The /api/register endpoint blindly binds all JSON fields from the request body to the user object without filtering or whitelisting allowed properties. By including an extra score field in the registration request body, an attacker can set their own leaderboard score to an arbitrary value (e.g., 10000) at account creation time, bypassing the intended game mechanics entirely.

Impact

- Attackers can manipulate internal fields such as score, role, isAdmin, or any other object property.
- Leaderboard integrity is completely compromised — any user can register with a maximum score.
- If role or admin fields are also unprotected, this can lead to privilege escalation.
- Undermines application business logic and fairness of any scoring system.

Steps to Reproduce

20. Intercept a POST /api/register request in Burp Suite Proxy.
21. In the request body, add an extra field: "score": 10000 alongside username and password.
22. Forward the request. Registration succeeds and the user is created with score=10000.
23. Navigate to GET /api/scores. The new user appears at the top of the leaderboard with 10000 points.

Solution

- Implement a strict allowlist of accepted fields on every input endpoint. Only bind explicitly permitted fields (e.g., username, password, email).
- Never use direct object/model mapping from raw user input in registration or update flows.
- Use Data Transfer Objects (DTOs) or input validation schemas to filter unexpected properties.

Flag: flag{br0k3n_oBj3cT_Pr0p3rTy_L3v3L_Auth0RiS4Ti0N}

Evidence Screenshots

The first screenshot shows the 'Request' tab in Burp Suite. The request is a POST to http://localhost:3000/register. The body is a JSON object: {"username": "mango128", "password": "mango128", "score": 10000}. The second screenshot shows the 'Response' tab. The response is a 200 OK from http://localhost:3000. The body is a JSON object: {"status": "success", "scores": [{"username": "test000", "score": 10000}, {"username": "mango128", "score": 10000}, {"username": "escap55", "score": 10000}]}. Both screenshots have red boxes highlighting the JSON bodies.

6. Server-Side Request Forgery (SSRF) [HIGH]

Vulnerability	Server-Side Request Forgery (SSRF)
OWASP API Category	API7:2023 — Server Side Request Forgery
Risk Factor	HIGH
Flag	No flag for this finding

Description

The /api/addNoteWithLink endpoint accepts a user-supplied URL parameter and makes a server-side HTTP request to that URL to fetch content for the note. There is no validation or allowlist of permitted domains. By providing a Burp Collaborator URL as the value, the server fetches the external URL and the request is received by the Collaborator server, confirming outbound SSRF. The server also stores the fetched HTML content (including injected payloads) in the note, suggesting additional stored XSS impact.

Impact

- Attackers can force the server to make requests to internal network resources (AWS metadata, internal APIs, database admin interfaces).
- Can be used to bypass firewall rules and access services not exposed externally.
- The server-fetched content is stored and reflected, enabling stored XSS.
- DNS/HTTP callbacks confirm outbound connectivity and can be used for data exfiltration.
- Full SSRF to internal network pivot in cloud environments.

Steps to Reproduce

24. Generate a Burp Collaborator payload URL (e.g., <http://6qxwf20kqvsa8o4txs2o1ov2ctik6auz.oastify.com>).
25. Send a POST `/api/addNoteWithLink` request with body: `{ "url": "<collaborator_url>" }`.
26. The server returns HTTP 200 with message: Successfully added a note and the note field shows the fetched HTML content.
27. In Burp Collaborator, DNS and HTTP interactions are received from the server IP, confirming SSRF.

Solution

- Implement a strict allowlist of permitted domains/IPs for any server-side URL fetch functionality.
- Block requests to private IP ranges (10.x.x.x, 172.16.x.x, 192.168.x.x, 169.254.x.x) and localhost.
- Do not store or reflect externally fetched content without sanitization.
- Disable or strictly control outbound HTTP from the application server.

Flag: No flag for this finding

Evidence Screenshots

[illegible]

7. Unrestricted File Upload / File Size Validation Bypass [HIGH]

Vulnerability	Unrestricted File Upload — Missing File Size Validation
OWASP API Category	API8:2023 — Security Misconfiguration
Risk Factor	HIGH
Flag	flag{file_size_is_important}

Description

The `/api/profile/upload` endpoint accepts file uploads for user profile pictures but does not enforce any restriction on file size. During testing, a file named `60Mb.jpg` (approximately 60.22 MB) was uploaded successfully and the server responded with HTTP 200 OK, confirming the file was stored without any size validation. The server returned a flag in the response body, confirming that file size validation is entirely absent. This vulnerability can be exploited to exhaust server storage, cause denial of service, and potentially trigger other server-side issues related to oversized file handling.

Impact

- Attackers can upload arbitrarily large files, consuming server disk space and causing storage exhaustion.
- Oversized file uploads can trigger denial of service (DoS) conditions on the server.
- Absence of size limits may allow bypassing other upload restrictions in combination with other vulnerabilities.
- Server processing of large files (e.g., image resizing, parsing) can cause CPU exhaustion.
- No enforcement of acceptable upload policy violates application security baseline.

Steps to Reproduce

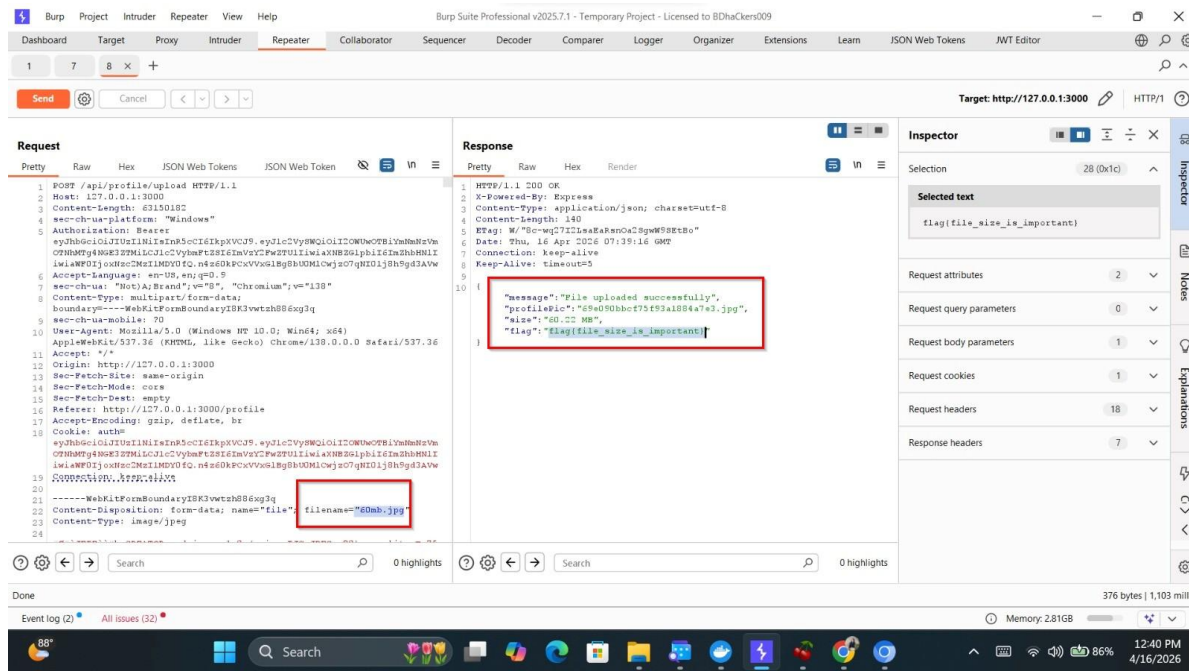
28. Log in to the application and obtain a valid JWT token.
29. Navigate to the profile upload functionality and capture the POST `/api/profile/upload` request in Burp Suite Repeater.
30. Replace the uploaded file with a large file (e.g., a 60 MB JPEG named `60Mb.jpg`) in the multipart form-data body.
31. Forward the request. The server responds with HTTP 200 OK and a success message.
32. Observe the response body: it confirms the file was uploaded (reporting size as ~60.22 MB) and returns the flag: `flag{file_size_is_important}`.

Solution

- Enforce a strict maximum file size limit on all upload endpoints (e.g., 2–5 MB for profile pictures).
- Validate file size both on the client side and server side — client-side checks alone are bypassable.
- Return HTTP 413 Request Entity Too Large when the size limit is exceeded.
- Implement overall request body size limits at the web server and application middleware level.
- Log and alert on unusually large upload attempts.

Flag: `flag{file_size_is_important}`

Evidence Screenshots



8. Improper Inventory Management [HIGH]

Vulnerability	Broken Function Level Authorization (BFLA)
OWASP API Category	API5:2023 — Broken Function Level Authorization
Risk Factor	HIGH
Flag	flag{a553Ts_m4N4g3m3NT_g0n3_wR0ng}

Description

The application does not properly manage and restrict its API inventory. During testing using Burp Suite, multiple endpoints such as /challenges, /api/*, and other accessible routes were discovered without proper documentation or access control. Some endpoints exposed internal data, including sensitive information embedded in response fields (e.g., flags inside descriptions), indicating that the API inventory is not properly maintained or secured. This lack of proper inventory management allows attackers to discover and interact with endpoints that may not be intended for public use, increasing the attack surface of the application.

Impact

- Attackers can discover undocumented or hidden endpoints.
- Exposed endpoints may leak sensitive information (e.g., internal data, flags).
- Increases attack surface, making exploitation easier.
- Can lead to further vulnerabilities like BOLA, BFLA, or data exposure.
- Helps attackers map the entire API structure for advanced attacks.

Steps to Reproduce

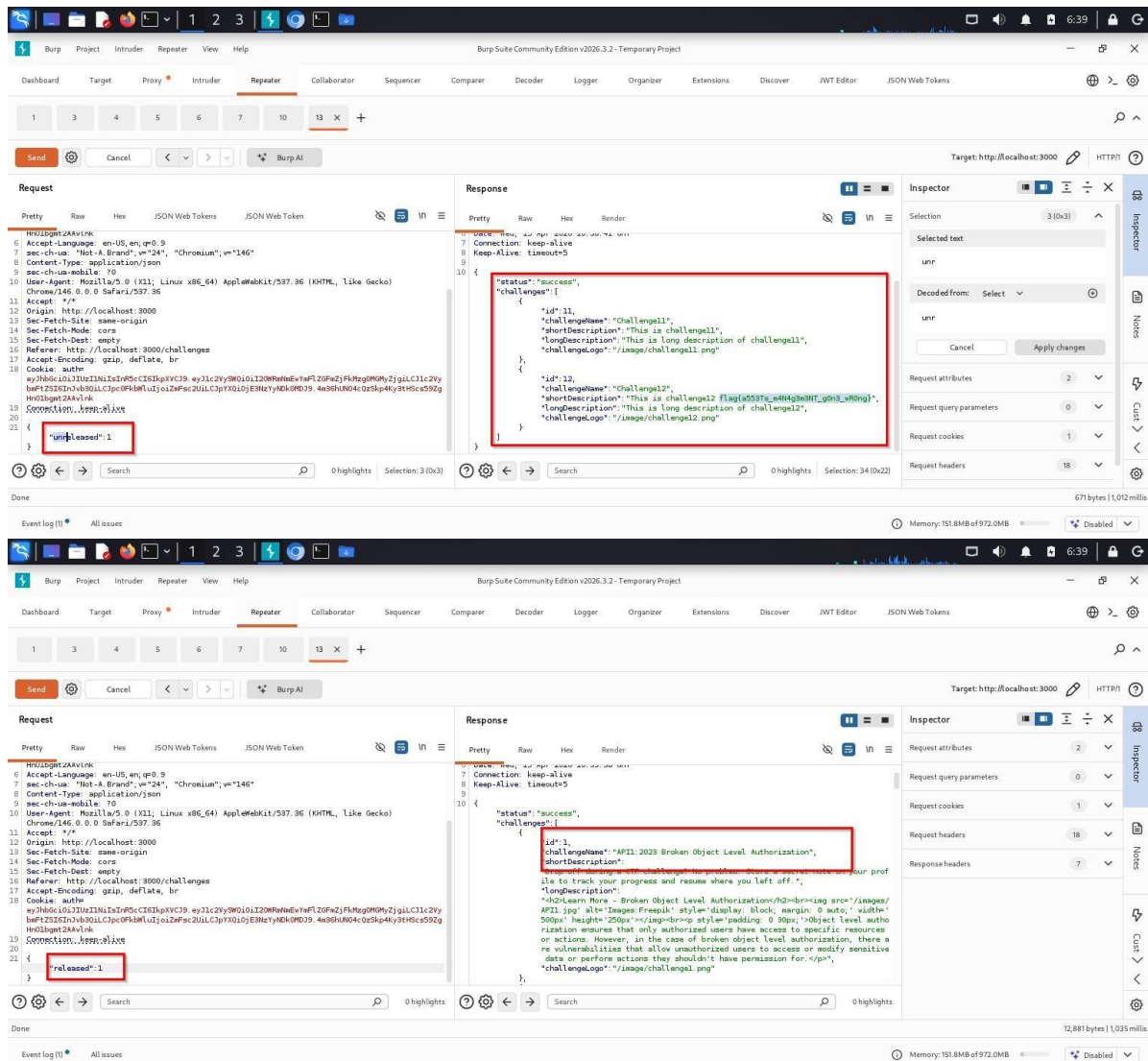
- Intercept API traffic using Burp Suite.
- Manually explore endpoints such as /challenges, /api/*, and other routes.
- Observe that multiple endpoints are accessible without restriction or proper documentation.
- Analyze responses and identify exposure of internal or sensitive data (e.g., flags in descriptions).
- Confirm that these endpoints are accessible without authentication or proper authorization checks.

Solution

- Maintain a proper and updated inventory of all API endpoints.
- Remove or disable unused, deprecated, or test endpoints.
- Restrict access to sensitive endpoints using authentication and authorization.
- Implement API versioning and proper documentation.
- Use API gateways to monitor and control endpoint exposure.
- Regularly audit APIs to identify and eliminate unnecessary endpoints.

Flag: flag{a553Ts_m4N4g3m3NT_g0n3_wR0ng}

Evidence Screenshots



MEDIUM FINDINGS

9. Missing Rate Limiting [MEDIUM]

Vulnerability	Missing Rate Limiting on Critical Endpoints
OWASP API Category	API4:2023 — Unrestricted Resource Consumption

Risk Factor

MEDIUM

Flag

No flag for this finding

Description

The application does not implement any rate limiting or account lockout mechanism on the /api/login endpoint. During testing using Burp Suite Intruder, multiple password attempts were sent rapidly for a single user account. The server did not block or throttle requests, and after multiple attempts, the correct password (at request #51) successfully authenticated the user and granted access. This confirms that the application is vulnerable to brute-force attacks due to missing rate limiting and account protection controls.

Impact

- Enables brute-force attacks to guess user passwords.
- Attackers can gain unauthorized access to accounts.
- No protection against repeated failed login attempts.
- Increases risk of account takeover at scale.
- Can be automated to compromise multiple users.

Steps to Reproduce

38. Send a POST request to /api/login using Burp Suite Intruder.
39. Use a payload list of multiple passwords for a valid username.
40. Start the attack and observe responses.
41. Notice that: No 429 Too Many Requests is returned. No account lockout or delay occurs. After multiple attempts (e.g., 51st request), the correct password results in Status: 200 OK, Message: Authentication successful.
42. Confirm that the account is successfully accessed without any restriction.

Solution

- Implement rate limiting on login endpoints (e.g., max 5–10 attempts per minute).
- Add account lockout after multiple failed attempts.
- Introduce exponential backoff or CAPTCHA after repeated failures.
- Monitor and log suspicious login behavior.
- Return HTTP 429 Too Many Requests when limits are exceeded.

Flag: No flag for this finding

Evidence Screenshots

The screenshot displays the Burp Suite Intruder interface. The 'Results' tab is active, showing a table of requests and responses. Request 51 is highlighted with a red box, indicating a successful login. The 'Response' tab for request 51 is also shown, displaying a JSON response with 'status': 'success' and 'message': 'Authentication successful'.

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
45	Cyber Kill Chain	401	3			280	
46	MITRE ATTACK	401	5			280	
47	Zero Trust Security	401	4			280	
48	Encryption	401	3			280	
49	Encryption	401	4			280	
50	Encryption	401	3			280	
51	escape	200	5			758	Contains a JWT

```
POST /api/login HTTP/1.1
Host: localhost:3000
Content-Length: 45
sec-ch-ua-platform: "Linux"
Accept-Language: en-US,en;q=0.9
sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
Content-Type: application/json
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: */*
Origin: http://localhost:3000
Sec-Fetch-Site: same-origin
```

```
{
  "status": "success",
  "message": "Authentication successful."
}
```

LOW FINDINGS

10. Long Session Lifetime & Token Re-issue Without Invalidation [LOW]

Vulnerability	Excessive Session Lifetime / JWT Not Invalidated on Re-issue
OWASP API Category	API2:2023 — Broken Authentication
Risk Factor	LOW
Flag	No flag for this finding

Description

The application issues new JWT tokens upon subsequent logins but does not invalidate previously issued tokens. During testing, a previously generated JWT token was manually substituted into a request (as annotated in Screenshot 9: 'I CHANGE THE CURRENT TOKEN TO THE PREVIOUS GENERATED TOKEN') and the server accepted it as valid. This means a stolen or leaked token remains usable even after the user logs in again and receives a new token. Combined with the absence of short expiration times, this significantly extends the window of risk for any compromised credential.

Impact

- A stolen JWT token remains valid indefinitely (or until expiry) even if the user re-authenticates.
- Attackers who obtain a token via MITM, XSS, or log exposure can use it long after the user believes their session has changed.
- No forced logout or token revocation is possible without server-side blacklisting.

Steps to Reproduce

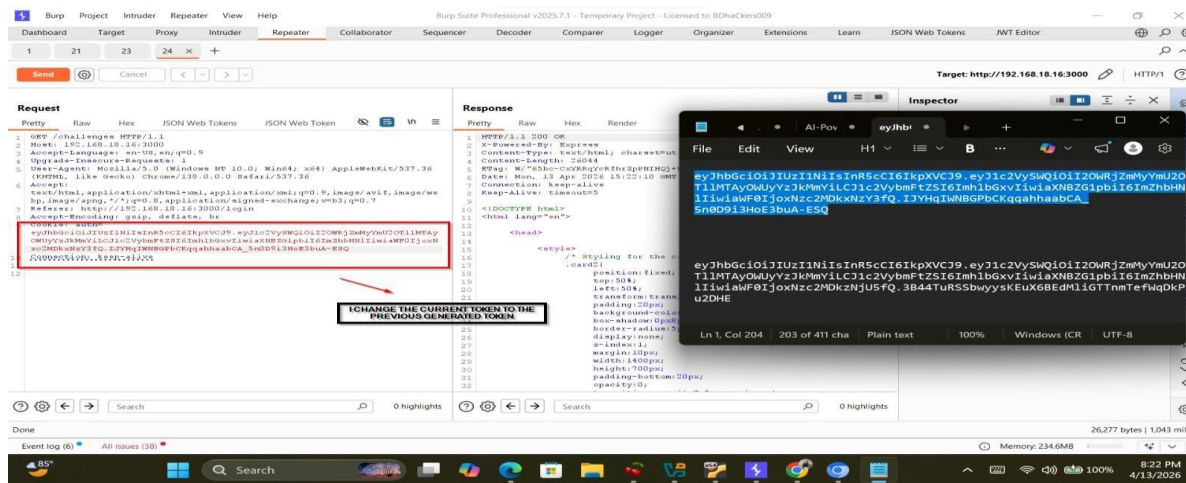
43. Log in to obtain JWT Token A. Note its value.
44. Log in again (or allow token rotation) to receive JWT Token B.
45. Use the old Token A in the Authorization header for a protected request.
46. Observe that the server accepts Token A as valid — confirming no invalidation on re-issue.

Solution

- Implement a server-side JWT blacklist or token versioning to invalidate previously issued tokens upon new login.
- Set short JWT expiration times (e.g., 15 minutes) with refresh token rotation.
- On logout, add the token JTI (JWT ID) to a revocation list and check it on every request.

Flag: No flag for this finding

Evidence Screenshots



Findings Summary Table

#	Vulnerability	Risk	Flag
1	BOLA / IDOR	CRITICAL	flag{bola_15_ev3rywh3r3}
2	NoSQL Injection	CRITICAL	flag{eZ_n0SQLi_pWn}
3	JWT Algo Confusion	CRITICAL	flag{St4cK_tR4c3_eRR0R}
4	Broken Function Level Auth (User Deletion)	CRITICAL	flag{n0_fUncTi0N_L3v3L_aUtH???
5	Mass Assignment	HIGH	flag{br0k3n_oBj3cT_Pr0p3rTy_L3v3L_Auth0RiS4Ti0N}
6	SSRF	HIGH	No flag
7	Unrestricted File Upload	HIGH	flag{file_size_is_important}
8	Improper Inventory Mgmt (BFLA)	HIGH	flag{a553Ts_m4N4g3m3NT_g0n3_wR0ng}
9	Missing Rate Limiting	MEDIUM	No flag
10	Long Session / No Token Invalidation	LOW	No flag

Report generated via manual security testing of DVAPI using Burp Suite Community/Professional Edition.